

RAPORT PROJEKTOWY

Wprowadzenia do baz danych – projekt końcowy	
Repozytorium projektowe	
Zespół: • Szymon Gładysz • Daniel Heczko • Mikołaj Dworak	Automatyka i Robotyka 2025/2026
	API: OPEN-METEO

Cel Projektu

Projekt ma na celu wykorzystanie wiedzy i umiejętności zdobytych podczas zajęć z zakresu projektowania oraz zarządzania bazami danych. W ramach realizacji zadania zespół zaprojektuje i utworzy bazę danych opartą na danych pozyskiwanych z wybranego, publicznie dostępnego interfejsu API.

Wykorzystywane Technologie

W projekcie wykorzystano język programowania Python, który został użyty do implementacji logiki aplikacji, komunikacji z API oraz obsługi operacji na bazie danych. Dodatkowo zastosowano język SQL, który posłużył do tworzenia struktury bazy danych oraz wykonywania zapytań umożliwiających przetwarzanie zgromadzonych informacji. W trakcie realizacji projektu wykorzystano również następujące biblioteki Pythona:

- pandas
- requests_cache
- retry_requests
- openmeteo_requests
- sqlalchemy
- psycopg2
- sys
- os
- tkinter
- datetime
- matplotlib

Wykorzystane biblioteki umożliwiły m.in. pobieranie danych z zewnętrznych źródeł, zarządzanie bazą danych oraz implementację interfejsu użytkownika i wizualizację danych.

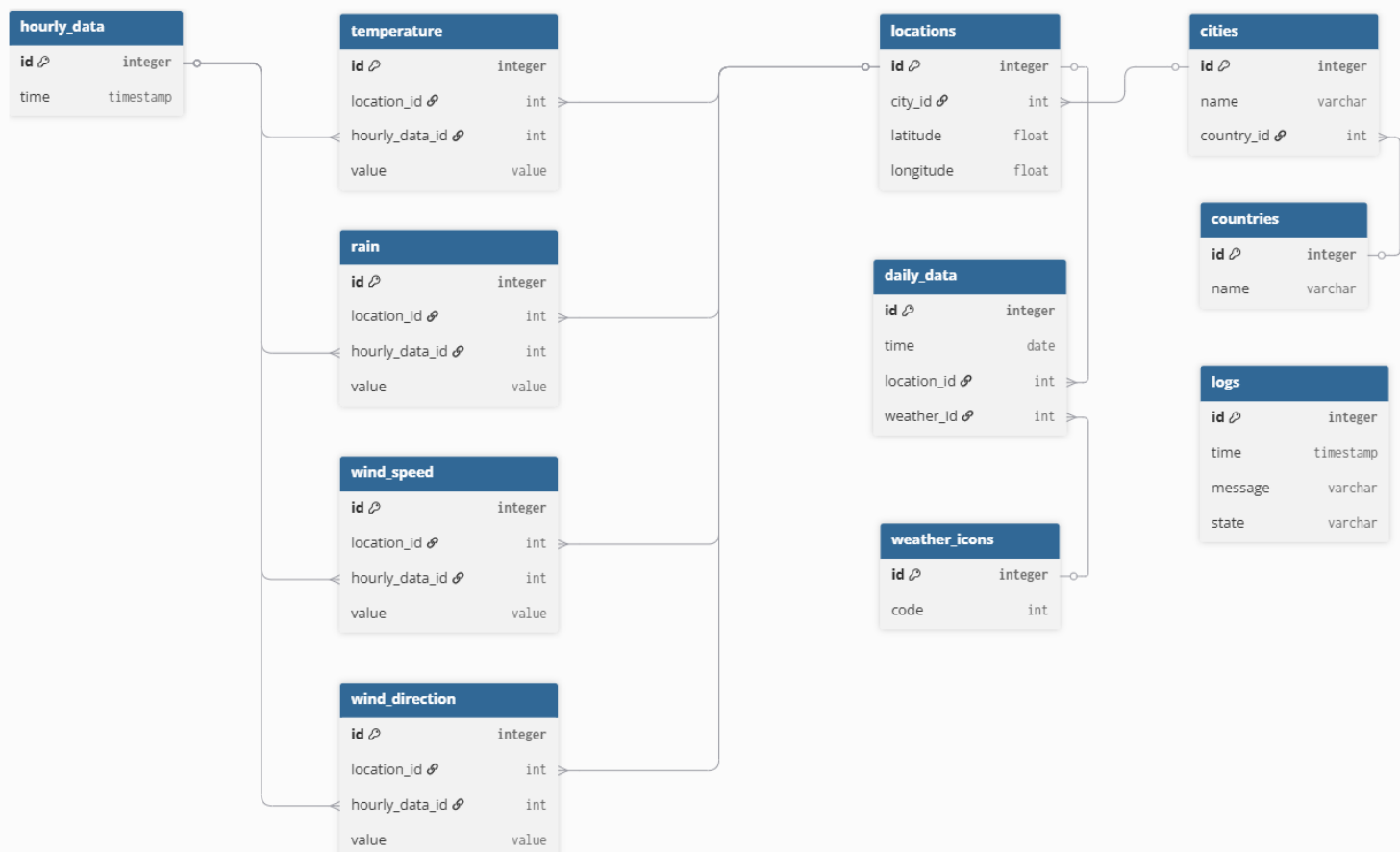
Struktura Projektu i Organizacja Projektu

Realizacja projektu została podzielona na kilka etapów, powiązanych z poszczególnymi plikami kodu źródłowego oraz ich funkcjonalnościami:

- *main.py* – główny plik wykonywalny projektu, odpowiedzialny za uruchomienie aplikacji.
- *import_data.py* – plik odpowiedzialny za pobieranie danych z publicznego API oraz zapisywanie ich do bazy danych.
- *init_db.py* – plik odpowiedzialny za inicjalizację bazy danych oraz tworzenie wymaganych tabel i relacji.
- *queries.py* – plik zawierający zapytania SQL do bazy danych wykorzystywane w aplikacji.
- *visualize.py* – plik z funkcjami pomocniczymi do wywoływania wykresów oraz map pogodowych (wizualizacja danych).

Projekt bazy danych

Opracowano diagram ERD przedstawiający strukturę bazy danych:



Struktura bazy danych składa się z tabel odpowiedzialnych za przechowywanie informacji o lokalizacjach, danych pogodowych oraz ich szczegółowych parametrach:

- Do przechowywania informacji o strukturze geograficznej wykorzystano tabele: **countries**, **cities** oraz **locations**. Tabela **countries** zawiera listę krajów, **cities** przechowuje miasta przypisane do krajów, natomiast **locations** definiuje konkretne lokalizacje wraz z współrzędnymi geograficznymi (szerokość i długość geograficzna).
- Tabela **daily_data** przechowuje podstawowe dane pogodowe dla poszczególnych dni, takie jak data oraz kod warunków pogodowych. Tabela ta łączy informacje o pogodzie z konkretną lokalizacją.
- Szczegółowe dane meteorologiczne zostały podzielone na osobne tabele: **hourly_data**, **temperature**, **rain**, **wind_speed** oraz **wind_direction**. Tabela **hourly_data** zawiera znaczniki czasu pomiarów, natomiast pozostałe tabele przechowują wartości odpowiednich parametrów pogodowych przypisanych do konkretnej lokalizacji i czasu.
- Tabela **weather_icons** przechowuje kody ikon odpowiadające warunkom pogodowym, natomiast tabela **logs** służy do zapisywania informacji systemowych dotyczących działania aplikacji.

Do wygenerowania poszczególnych tabel wykorzystano bibliotekę sqlalchemy, a poniżej zaprezentowano kod generujący przykładową tabelę **temperature**.

```
with engine.connect() as conn:
    conn.execute(text("""
        CREATE TABLE IF NOT EXISTS temperature (
            id SERIAL PRIMARY KEY,
            location_id INTEGER,
            hourly_data_id INTEGER,
            value FLOAT
        )
    """))
```

Następnie zaprojektowano relacje między tabelami w nowo utworzonej bazie. Poniższy kod tworzy relację pomiędzy tabelą **temperature** a tabelą **locations** (poprzez kolumnę `location_id` w `temperature` wskazującą na `id` w `locations`). Drugi fragment w tym samym ALTER dotyczy relacji pomiędzy tabelą **temperature** a **hourly_data** (poprzez `hourly_data_id`).

```
conn.execute(text("""
    ALTER TABLE temperature
    ADD CONSTRAINT fk_temperature_location
    FOREIGN KEY (location_id) REFERENCES locations(id)
    ON DELETE CASCADE,
    ADD CONSTRAINT fk_temperature_hourly_data
    FOREIGN KEY (hourly_data_id) REFERENCES hourly_data(id)
    ON DELETE CASCADE;
"""))
```

Pobieranie danych z API

W projekcie wykorzystano dane obejmujące okres ostatnich pięciu lat. Zbiory danych są aktualizowane automatycznie w interwale godzinowym, co zapewnia ich ciągłą synchronizację i spójność czasową.

Dane są pozyskiwane dla zdefiniowanego zestawu miast, obejmujących: Kraków, Londyn, Paryż, Tokio, Waszyngton, Rzym, Helsinki, Rio de Janeiro, Kair oraz New Delhi.

Operacje na bazie danych

Zapytania SQL zostały użyte do selekcji danych prezentowanych na wykresach. Są one zapisane np. w pliku queries.py. Przykładowe zapytania wykorzystane w projekcie:

- **Statystyki pogodowe (temperatura / wiatr / deszcz)**

```
SELECT
DATE_TRUNC('month', hd.time)::date AS time,
AVG(t.value) AS value
FROM temperature t
JOIN hourly_data hd ON t.hourly_data_id = hd.id
WHERE t.location_id = :location_id
AND hd.time::date BETWEEN :date_from AND :date_to
AND EXTRACT(DOW FROM hd.time) IN (:week_days)
GROUP BY DATE_TRUNC('month', hd.time)::date
ORDER BY time;
```

Zapytanie służy do pobierania danych pogodowych (np. średniej, wartości maksymalnej lub minimalnej) dla wybranego parametru pogodowego. Dane są pobierane z odpowiedniej tabeli (**temperature**, **wind_speed** lub **rain**) i łączone z tabelą **hourly_data**, która przechowuje informacje o czasie pomiaru. Dane są filtrowane według zakresu dat (**date_from – date_to**), wybranej lokalizacji (**location_id**) oraz dni tygodnia (**week_days**). Następnie wyniki są grupowane według wybranego interwału czasowego (rok, miesiąc lub dzień), co pozwala na analizę trendów w czasie.

- **Porównanie dwóch zmiennych pogodowych**

```
SELECT
DATE_TRUNC('month', hd.time)::date AS time,
AVG(t1.value) AS value1,
AVG(t2.value) AS value2
FROM hourly_data hd
JOIN temperature t1
```

```

ON t1.hourly_data_id = hd.id AND t1.location_id = :location_id
JOIN wind_speed t2
ON t2.hourly_data_id = hd.id AND t2.location_id = :location_id
WHERE hd.time::date BETWEEN :date_from AND :date_to
AND EXTRACT(DOW FROM hd.time) IN (:week_days)
GROUP BY DATE_TRUNC('month', hd.time)::date
ORDER BY time;

```

Zapytanie umożliwia jednoczesne pobranie i porównanie dwóch różnych parametrów pogodowych (np. temperatury i prędkości wiatru) w tym samym przedziale czasowym. Dane pochodzą z dwóch tabel (np. **temperature**, **wind_speed**) i są łączone na podstawie wspólnego identyfikatora czasu (**hourly_data**) oraz tej samej lokalizacji (**location_id**). Wyniki są agregowane w zadanym interwale czasowym (rok, miesiąc lub dzień), co pozwala na analizę zależności pomiędzy różnymi zjawiskami pogodowymi w czasie.

- **Dominujący kierunek wiatru (dla lokalizacji)**

```

SELECT
l.latitude,
l.longitude,
MODE() WITHIN GROUP (ORDER BY wd.value) AS wind_direction
FROM wind_direction wd
JOIN hourly_data hd ON wd.hourly_data_id = hd.id
JOIN locations l ON wd.location_id = l.id
WHERE hd.time::date BETWEEN :date_from AND :date_to
AND EXTRACT(DOW FROM hd.time) IN (:week_days)
GROUP BY l.id, l.latitude, l.longitude;

```

Zapytanie wyznacza dominujący (najczęściej występujący) kierunek wiatru dla każdej lokalizacji w określonym przedziale czasowym. Dane są pobierane z tabeli **wind_direction** i łączone z **hourly_data** (informacja o czasie pomiaru) oraz **locations** (współrzędne geograficzne). Funkcja statystyczna **MODE()** wybiera najczęściej występującą wartość kierunku wiatru dla danej lokalizacji. Wynikiem są współrzędne geograficzne wraz z dominującym kierunkiem wiatru.

- **Kody pogodowe dla lokalizacji (dziennie)**

```

SELECT
l.latitude,
l.longitude,
wi.code AS weather_code

```

```
FROM daily_data dd
JOIN locations l ON dd.location_id = l.id
LEFT JOIN weather_icons wi ON dd.weather_id = wi.id
WHERE dd.time = :target_date;
```

Zapytanie pobiera kody pogodowe dla wszystkich lokalizacji w wybranym dniu. Dane pochodzą z tabeli **daily_data**, która przechowuje codzienne informacje pogodowe dla lokalizacji. Następnie tabela ta jest łączona z **locations** (współrzędne geograficzne) oraz **weather_icons** (kody opisujące warunki pogodowe). Wynikiem zapytania jest zestawienie współrzędnych geograficznych wraz z przypisanym kodem pogody, co może być wykorzystane np. do wizualizacji mapy pogodowej.

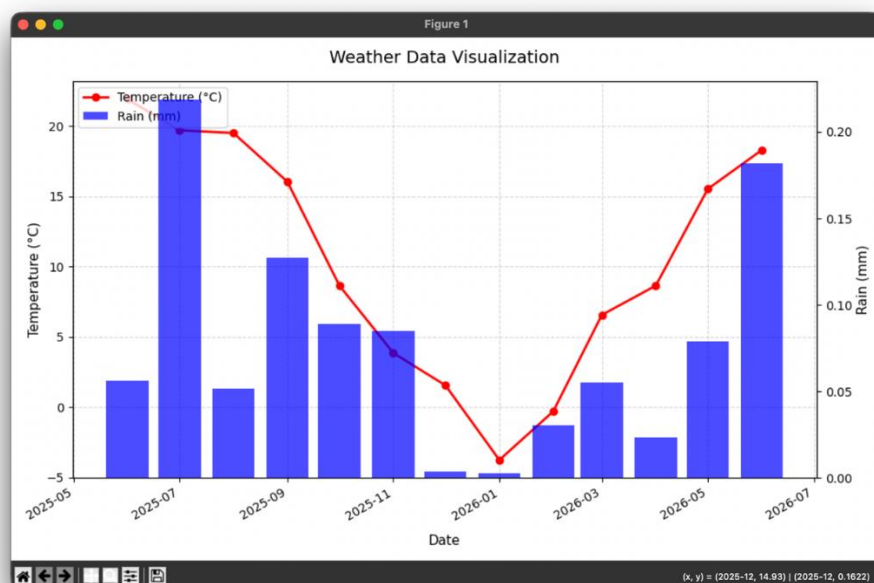
- **Pobranie wszystkich lokalizacji z współrzędnymi geograficznymi**

```
SELECT id, latitude, longitude FROM locations
```

Zapytanie pobiera z tabeli **locations** trzy kolumny: **id**, **latitude** oraz **longitude**. **id** – unikalny identyfikator rekordu (np. lokalizacji), **latitude** – szerokość geograficzna danej lokalizacji, **longitude** – długość geograficzna danej lokalizacji. Wynikiem zapytania jest lista wszystkich zapisanych lokalizacji wraz z ich współrzędnymi geograficznymi, bez żadnych filtrów ani dodatkowych warunków.

Wyniki

Działanie programu polega na generowaniu formularza służącego do wizualizacji danych pochodzących z bazy danych, z wykorzystaniem zapytań SQL. Projekt został wyeksportowany do pliku wykonywalnego. Przykładowy formularz projektowy wraz z wykresem wizualizującym dane oraz mapami pogodowymi:



Open-Meteo Data Visualizer

Visualization type

 &  Temperature & Rain

Time

Time interval

☐ Year ☒ Month ☐ Day

From

2025-06-22

To

2026-06-22

Location

Location

Kraków

Week days

☒ Monday

☒ Tuesday

☒ Wednesday

☒ Thursday

☒ Friday

☒ Saturday

☒ Sunday

NONE

Units

Temperature

☒ °C ☐ °F

Rain

☒ mm ☐ inch

Options (Temperature)

Statistic

☒ avg ☐ max ☐ min

Chart type

☒ Line ☐ Bar ☐ Scatter

Color

Choose color

#ff0000

Options (Rain)

Statistic

☒ avg ☐ max ☐ min

Chart type

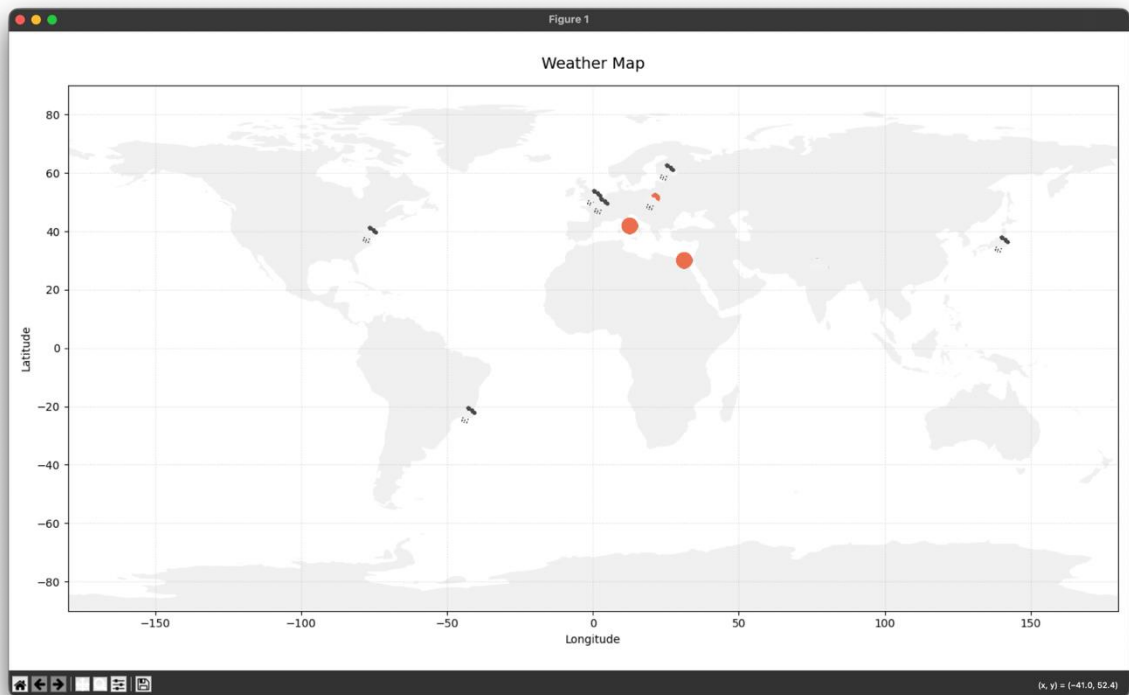
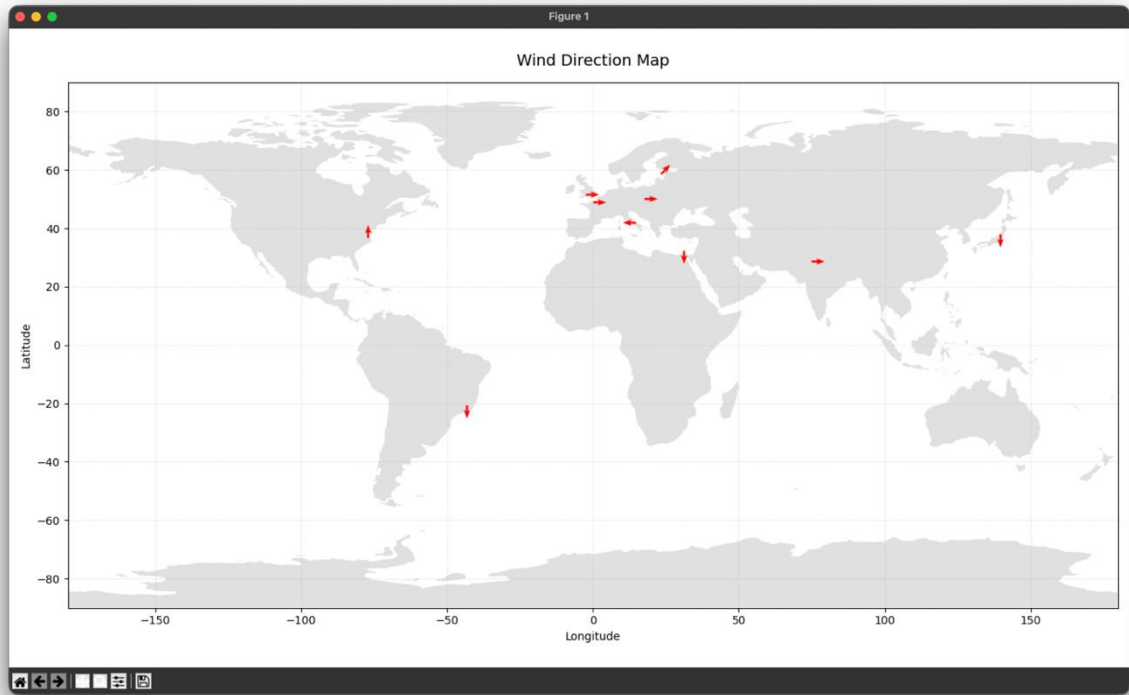
☐ Line ☒ Bar ☐ Scatter

Color

Choose color

#0000ff

Visualize data



Podział pracy

Daniel Heczko

- Lider zespołu
- Odpowiedzialny za pliki: main.py, visualize.py
- Refaktoryzacja kodu
- Interfejs i wizualizacja danych
- Raport projektowy

Mikołaj Dworak

- Odpowiedzialny za pliki: inport_data.py
- Zczytywanie danych z API
- Struktura bazy danych
- Raport projektowy

Szymon Gładysz

- Odpowiedzialny za pliki: init_db.py, queries.py, visualize.py
- Struktura bazy danych
- Inicjacja bazy danych
- Raport projektowy

Podsumowanie

Za cel projektu przyjęliśmy zaprojektowanie i implementację bazy przechowującej dane metrologiczne pobierane z publicznego API, Open-Meteo. W ramach realizacji projektu stworzyliśmy kompletny system obejmujący pobieranie, przetwarzanie oraz wizualizację danych z API. Aplikację zrealizowaliśmy w języku Python z wykorzystaniem bibliotek wspierających komunikację z API, obsługę baz danych i wizualizację danych.

Zaprojektowana baza została zaprojektowana na relacyjnej strukturze obejmującej dane geograficzne oraz historyczne i aktualne dane pogodowe. Dzięki zastosowanemu podziałowi na specjalizowane tabele zachowaliśmy spójność danych i umożliwiliśmy ich efektywne przetwarzanie.

Dane pogodowe dla wybranych lokacji są pobierane automatycznie i aktualizowane w godzinnych interwałach

Stworzone zapytania SQL pozwalają na porównywanie danych w wybranym przedziale czasowym, wyznaczenie dominującego kierunku wiatru i pobranie kodów pogodowych oraz lokacji.

Projekt zakończył się sukcesem i spełnił założone założenia, umożliwiając efektywne przechowywanie i analizę danych pogodowych oraz ich dalsze wykorzystanie w celach analitycznych i edukacyjnych.